# PYTHON
# RECALL

Digital Learning Hub

# Python Variables

- Create a new file to start with our refresh >> **variables.py**

- Creating variables syntax:

  **varName = value**

  **print(varName)** // to display a variable's value

- Data Types: Integers, floats, Complex **[Numeric Types]**

  Strings, Lists, Tuples, range **[Sequences Types]**

  Dict, sets, frozenset **[Mapping and Set Types]**

  **True False [Boolean Type / Logical Type]**

  **None [NoneType]**

- Arithmetic operations :

| | |
|---|---|
| + | **Addition** |
| - | **Subtraction** |
| * | **Multiplication** |
| ** | **Exponentiation** |
| / | **Division** |
| // | **Floor Division** |
| % | **Modulus** |

**Python Comments ->**

# Python Comments

To write comments throughout the code in python we have 2 Ways of doing so:

- Single Line comment:

  *# This is a Single line comment*

- Multiple Line Comment:

```
"""
  This is a Multiple line of comments
  This is the Second line of comment
"""
```

**or**

```
'''
  This is a Multiple line of comments
  This is the Second line of comment
'''
```

**String formatting ->**

# String Formatting

- Using the **f-String** example:

```
name = "Vanessa"
print(f"Hello {name}")
```

- Using a **String Format** example:

```
message = "Hi {}"
name = "Vanessa"
print(message.format(name))
```

**Getting the User Input ->**

# Getting the User Input

- User input Example:

```
name = input("What is your name?")
age = input("How old are you?")
```

** The input function always stores the value given by the user as a String and assigns it to the variable, which is why mostly when we need the data given for an arithmetic operation we need to convert the data type as the following:

```
age = int(input("How old are you?"))
```

** you can use **float()**, and **str()** too

**Lists in Python ->**

# Lists in Python

- Lists are just collection of data, you can create a list that contains different data types and you cannot access and index out of range

- List example: my_list = [80, 90, '50', 70.45]
  indexes 0 1 2 3
- To select one element from the list you need to refer to it by its index such as:
  print(my_list[2]) >> **it will Display 50**

- To update a value in the list: my_list[2] = **10**
- You can select the last element of any list by using the -1 index as the following: my_list[-1]

- To count how many elements you have in a list : len(my_list)
- To slice the list : my_list[0:2] *we start at index 2 and we stop at index 2 but index 2 is not included*

- To add an element to the list :
  my_list.append(35) -- *it will always add the element at the end of the list*
  my_list.insert(2, 35) -- *it will add the element at the index 2 in the list*

- To delete an element from the list:
  my_list.pop() -- *it will always delete 1 element from the end of the list*
  my_list.remove(35) -- *it will delete the value 35 from the list*

**Sets in python ->**

# Sets in python

- Sets are similar to Lists but they are not ordered and they cannot contain duplications
- Set example : **mySet = {20, 15, 25, 35, 20, 15}**

- To Display all the elements in a set you need to use a loop
- You cannot use indexes with a set

- You can remove an element from the set as the following: **mySet.discard(20)**
- You can empty a set : **mySet.clear()**
- You can add an element to the set : **mySet.add(2)**
- You can add mor than 1 element to a set: **mySet.update([7, 8])**

**Tuples in Python ->**

# Tuples in Python

- Tuples are like lists, ordered but they are unchangeable
- Tuple example : **myTuple = (1, 2, 3, 4, 5)**
- Select 1 element from the Tuple : **myTuple[2]**

**Boolean and Operators ->**

# Boolean and Operators

- A Boolean is a logical data type that takes one of these 2 values : **True** or **False**
- Example of a boolean: **like_coffee** = **True**

- **Comparison Operators:**

  >> 1 == 2 – the result of that comparison will result with **False**
  >> 1 != 2 – the result of that comparison will result with **True**
  >> 1 > 2 – the result of that comparison will result with **False**
  >> 1 < 2 – the result of that comparison will result with **True**
  >> 1 >= 1 – the result of that comparison will result with **True**
  >> 1 <= 2 – the result of that comparison will result with **Ture**

- **Logical Operators:**

  >> 1 > 2 **and** 5 < 7 – The result will be **False**
  >> 1 > 2 **or** 5 < 7 – The result will be **True**
  >> **not(**1 == 1) – The result will be **False**

**Control flow conditional statements ->**

# Control flow conditional statements

- Take this example:

```
x = 1
if x == 1:
    print("X is equal to 1")
elif x == 2:
    print("X is equal to 2")
else:
    print("The value of X is unknown")
```

- You can use the **continue** and **break** statements with you conditions

**Control Flow loops statements ->**

# Control flow loops statements

- Take the following example:

```
my_list = [1, 2, 3, 4, 5]

1) for x in my_list:
        print(x)

2) for x in range(3, 6):
        print(x)

3) i = 0
   while i < 5:
      print(my_list[i])
```

Dictionaries in Python ->

# Dictionaries in Python

- A Dictionary is a set of **key – value** pairs as the following:

```
my_dict = {
  'username' : "vanessa",
  'age' : 25
}
```

- Display a certain value from that Dictionary: `print(my_dict.get('username'))`

- Adding a key to the Dictionary : `my_dict['favorite_food'] = "Chicken"`
- Delete a key from the Dictionary : `my_dict.pop('age')`
- Empty a Dictionary: `my_dict.clear()` **or** `del my_dict`

- Copy a Dictionary : `my_dict2 = my_dict.copy()`

- Loop in a Dictionary:

```
for x, y in my_dict.items():
    print(x, y)
```

**Functions in Python ->**

# Functions in Python

- Functions are way to isolate code, and only be able to do certain aspects of code when we want that functionality o be called. Examples:

```
1) def my_function():
       print("Hello world!")

   my_function()

2) def my_function(name):
       print(f"Hello {name}!")

   my_function(name="vanessa")

2) def multiply(a, b):
       return a * b

 result = multiply(3, 10)
```

Imports in Python ->

# Imports in Python

- We can create a python file that contains functions we create isolated and import them in our main file where we are typing the code.
- We can also import functions from python modules ready to be used by anyone
- We can import using methods:

```
>> import math
>> from math import pi
>> from math import *
>> import math as mat
```

Object Oriented Programming in Python ->

# OOP in Python

- Object Oriented Programming is a programming paradigm based on the concepts of objects which can contain data and code

- OOP benefits:
  - >> Scalability
  - >> Efficiency
  - >> Reusability

- We can define objects by **behavior** or by **state**
- Example:

```
class Cat:
        legs : int = 4
        ears: int = 2
        type : str = "Angora Turkish"
        age : int = 5
        color : str = "White"
cat = Cat() # cat object
cat.legs
cat.ears
```

**Small OOP demo Given ->**

# Small OOP demo Given

We will create a small game where we can create enemies that fight each other

- **Enemies that can fight each other**
- **Different types of enemies (zombie, ogre etc..)**
- **Each enemy have different powers and health points and attack damage**

We will be implementing the four pillars of OOP:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

1. We will create an Enemy Object: ***[we will create is as a class]***

   >> Name / Type of Enemy
   >> Health points
   >> Attack damage

**Demo Structure ->**

# Demo Structure

- Create a file **<<enemy.py>>:**

```
class Enemy: [Bad Practice]
        type_of_enemy : str
        health_points: int = 10
        attack_damage : int = 1
```

- In the **<<main.py>>** file:

```
from Enemy import *

zombie = Enemy()

 zombie.type_of_enemy = 'Zombie'

print(f"{zombie.type_of_enemy } has {zombie.health_points} health points and can
do attack of {zombie.attack_damage }")
```

**Good Practice using constructor ->**

# Good Practice using constructor

- Good practice for the Enemy Class:

```
class Enemy:
    def __init__(self,enemy_type : str, health : int = 10, damage : int = 1):
        self.type_of_enemy: str = enemy_type
        self.health_points: int = health
        self.attack_damage = damage
```

** <<self>> refers to the current class Object

Abstraction in Python ->

# Abstraction in Python

- Abstraction means to **hide the implementation** and only **show necessary details** to the user

- We will create more methods in our enemy class:

```
def talk(self):
        print(f'I am a {self.type_of_enemy}. Be prepared to fight!')

def walk_forward(self):
        print(f' {self.type_of_enemy} Moves closer to you.')

def attack(self):
        print(f' {self.type_of_enemy} attacks for {self.attack_damage} damage.')
```

***Use these methods in the main.py***

**Encapsulation in Python ->**

# Encapsulation in Python

- Encapsulation means bundling of Data

- We will use encapsulation to prevent the user to change the type of the enemy after being initialized. For that we need to change the public attributes into private attributes by adding **double underscore.** Such as:

```
class Enemy:
    def __init__(self,enemy_type : str, health : int = 10, damage : int = 1):
        self.__type_of_enemy: str = enemy_type
        . . .
```

- When turning an attribute from public to private, wee no longer can change it directly, this is why Encapsulation always comes with a **SETTER** to set a value and a **GETTER** to get the value such as:

```
def get_type_of_enemy(self):
        return self.__type_of_enemy

def set_type_of_enemy(self, type_of_enemy):
        self.__type_of_enemy = type_of_enemy
```

**Inheritance in Python ->**

# Inheritance in Python

- Inheritance – Process of acquiring properties from one class to another class. It creates a type of hierarchy between classes (Parent – Child classes)

- Example:

```
class Animal: -- Parent Class
        weight : int
        color : str
        age : int
        animal_type : str

        def eat(self):
                print('Animal Eating')


        def sleep(self):

            print('Animal Sleeping')
```

```
class Dog(Animal):
        ** All Animal Attributes **
        can_shed : bool
        domestic_name : str
        ** All Animal Methods **

        def talk(self):
            print('Bark')
        def eat(self): -- Overriding a method
            print('Chews on Bone')
```

```
dog = Dog()
dog.talk() # Bark
dog.eat() # Chews on Bone
```

**Self vs super in Python ->**

# Self vs Super in Python

- **Self** is used to refer to the current object that is created or being instantiated, while **super** is used to refer to the Parent class
- **Self** is used when there is a need to differentiate between instance variables & parameters with the same name while **super** is used to call the Parent class methods and/or constructors
- Takes a **Person** and **Student** classes example, where the student class inherits from the Person class, and see how their constructors would be created:

```
def __init__(self, name, age): - - > Constructor for the Person Class
        self.name = name
        self.age = age

def __init__(self, name, age, degree): - - > Constructor for the Student Class
        super(). __init__(name = name , age = age)
        self.degree = degree
```

**Implementing Inheritance in our demo ->**

# Implementing Inheritance in our demo

- The **Enemy** will be our **Parent** class and we will create **2 Children** classes **Zombie** and **Ogre**
- Zombie Class:

```
class Zombie(Enemy):
        def __init__(self, health_points, attack_damage):
            super(). __init__(
                        type_of_enemy = 'zombie' ,
                        health_points= health_points ,
                         attack_damage= attack_damage
             )

        def talk(self): # Method Overriding
                        print('Grumbling … ')
        def spread_diseaase(self):
                        print('Zombie is trying to spread infection')

class Ogre(Enemy):
        def __init__(self, health_points, attack_damage):
            super(). __init__(
                        type_of_enemy = 'Ogre' ,
                        health_points= health_points ,
                         attack_damage= attack_damage
             )

        def talk(self): # Method Overriding
                        print('Ogre is slamming Hand all around')
```

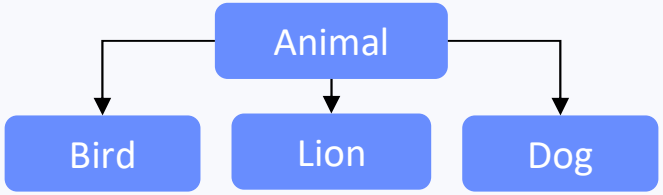
**Polymorphism in Python ->**

# Polymorphism in Python

- **Polymorphism** means to have many forms. It means changing an object at a specific runtime

- Take the Animal and Dog classes as example:

```
zoo = Animal[]

dog = Animal()
dog2 = Dog()
bird = Bird()
lion = Lion()

zoo.append(dog)
zoo.append(dog2)
zoo.append(bird)
zoo.append(lion)

for animal in zoo:
        animal.talk()
```




*# Each have a method*

**Implementing polymorphism in our demo ->**

# Implementing polymorphism in our demo

- We will create a new battle function within our **<<main.py>>** file. This function will use our Enemy **talk** and **attack** methods such as the following:

```
def battle(Enemy e):
        e.talk()
        e.attack()
```

- It will be used as the following:

```
zombie = Zombie(10,1)
ogre = Ogre(20,3)

battle(zombie)
battle(ogre)
```

**Improvements with our demo ->**

# Improvements with our demo

- We will give our Enemies a special attack

  >> If the Enemy does not have a special attack, we print "Enemy does not have a special attack"

  >> We will add code in the Enemy (Parent) and both Zombie and Ogre

- In the **Enemy** Class:

```
def special_attack(self):
        print('Enemy has no special attack')
```

- In the **Zombie** Class:

```
def special_attack(self):
        did_special_attack_work = random.random() < 0.50
        if did_special_attack_work :
                self.health_points += 2
                print('Zombie regenerated 2 HP!')
```

- In the **Ogre** Class:

```
def special_attack(self):
        did_special_attack_work = random.random() < 0.20
        if did_special_attack_work :
                self.attack_damage += 4
                print('Ogre attack has increased by 4!')
```

**Improvements with our demo ->**

# Implementing polymorphism in our demo

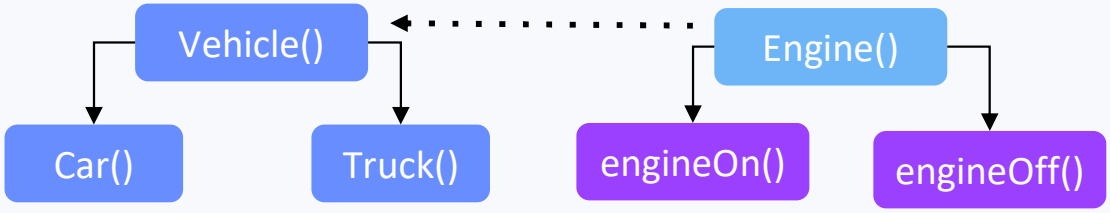
- Update the **battle** function as the following:

```
def battle(e1: Enemy , e2: Enemy):
        e1.talk()
        e2.talk()

        while e1.health_points > 0 and e2.health_points > 0 :
                e1.special_attack()
                e2. special_attack()

                e2.attack()
                e1. health_points  -= e2. attack_damage

                e1.attack()
                e2. health_points  -= e1. attack_damage
        if  e1. health_points > 0 :
                print("Enemy 1 Wins!")
        else:
                print("Enemy 2 Wins!")
```

Composition in python ->

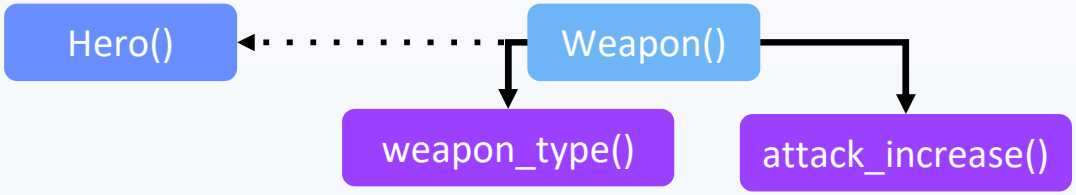
# Composition in python

- Composition is a way to create objects made up of other objects. In composition a class contains one or more objects of another class as instance variables, It provides layers of functionality to the object. It is known as **HAS-A** relationship




- To implement Composition in our demo:
  - >> We will create a Hero class
  - >> We will create a new Weapon class

  >> Our Hero will have a **HAS-A (composition)** relationship with our Weapon class




**Implementing Composition in our demo ->**

# Implementing Composition in our demo

- In our **Weapon** class:

```
class Weapon:
        def __init__(self, weapon_type, attack_increase):
                                self.weapon_type = weapon_type
                        self.attack_increase = attack_increase
```

- In our **Hero** class:

```
class Hero:
        def __init__(self, health_points, attack_damage):
                                self.health_points health_points
                        self.attack_damage = attack_damage
                        self.is_weapon_equipped = False
                         self.weapon : Weapon = None

         def equipWeapon(self):
                        if self.weapon is not None and not self.is_weapon_equipped:
                                self.attack_damage +=  self.weapon.attack_increase
                                 self.is_weapon_equipped = True
```

**Implementing Composition in our demo ->**

# Implementing Composition in our demo

- In the **<<main.py>>** file create a **hero_battle function**

```
def hero_battle(hero: Hero , enemy : Enemy ):
        while hearo.health_points > 0 and enemy.health_points > 0 :
                enemy.special_attack()
                enemy.attack()

                hero. health_points  -= enemy. attack_damage

                hero.attack()
                enemy. health_points  -= hero. attack_damage

        if  hero. health_points > 0 :
                print("Hero Wins!")
        else:
                print("Enemy 2 Wins!")
```

- In the **<<main.py>>**:

```
hero = Hero(10,1)
weapon = Weapon('Sword',5)
hero.weapon = weapon
hero.equip_weapon()
hero_battle(hero, zombie)
```

**Introduction to FastAPI ->**

Digital
Learning
Hub

# Develop your digital skills

The mission of the Digital Learning Hub is to help reduce the digital skills gap in Luxembourg.
We do so by offering in-person, affordable courses in various fields of IT and related sectors.

**Introduction to FastAPI Web Development with Python**

**Instructor : Vanessa Al Daham**
**Course Language: English**

Welcome 👋

# Let’s get to know each other

## What’s your name?

Where do you work? Do you have any experience in it?

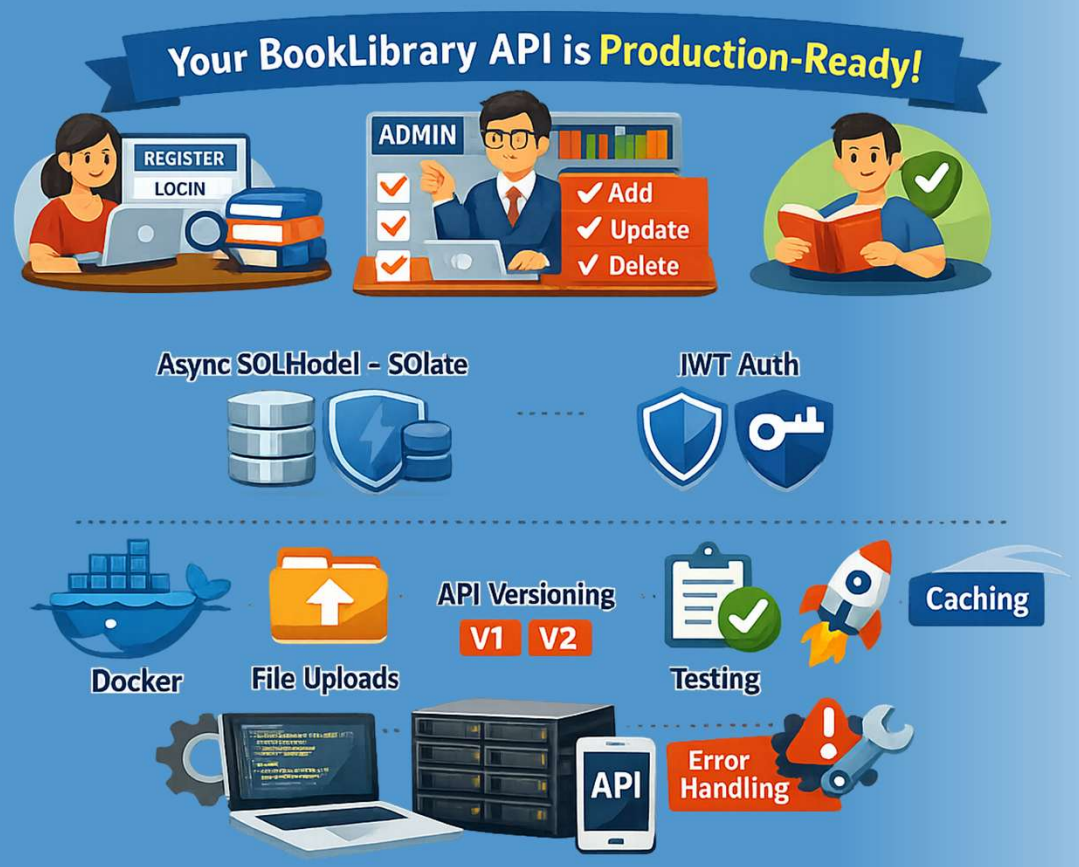
What are your expectations of this course?

**Intermediate** Course

# Foundations & Core

# FastAPI

Digital Learning Hub

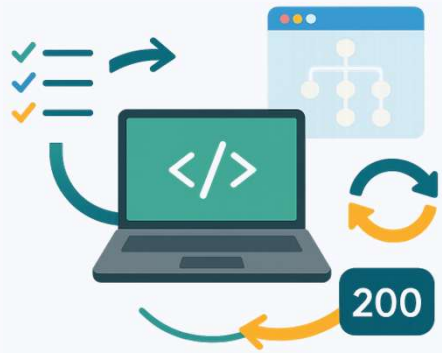



By the end of this course, every learner will have a complete, production-ready **BookLibrary** API where:

- Anyone can register, log in, and search books.Admin users can add/update/delete books (with cover image upload)
- Regular users can borrow books (protected route + background task)
- The app uses **async SQLModel + SQLite**, **JWT** auth with **roles**, **file uploads + static serving**, custom **error handling**, **versioning**, **tests**, **caching**, and a **Docker deployment**

# Foundation & Core

FastAPI is a modern Python web framework that lets you build high-performance APIs using ordinary Python functions and type hints.
It automatically generates interactive documentation (Swagger UI at **/docs** and ReDoc at **/redoc**)




- **Project Setup:**
  - Create a virtual environment : **python -m venv venv**
  - Activate it and install: **pip install fastapi uvicorn pydantic-settings**
  - **.env** file **+ pydantic-settings** keeps configuration (**APP_NAME**, **SECRET_KEY**) clean and safe — never hard-code secrets

- **First Endpoint:** The **@app.get("/")** decorator turns a function into a web-accessible route. When a request arrives, FastAPI runs your function and returns the result as JSON

**Task Time ->**

# Task Time

1. Create the main project folder **<<book_Libary>>**
2. Create the **<<app>>** folder and the files in it, **main.py** and **config.py**

3. In the **<<config.py>>:**
   **>>** Use the BaseSettings object from the pydantyic-settings module to create the class Settings
   **>>** Set the **APP_NAME**, **SECRET_KEY** and **DATABASE_URL**, they must be Strings
   **>>** The **.env** file must be loaded

4. Create the **.env** file and type the variables needed

5. In the **<<main.py>>:**
   **>>** Create the **app**, and set the App **title**, **description** and **version**
   **>>** Create the root route **GET /**, that triggers an async function. The function should return a welcoming message to the app
   **>>** Add a simple **GET /health** that returns server status

Path Operations . . . ->

## Path Operations, Request/Response Lifecycle, Status Codes:

- **Path Operations = the URLs your API answers (@app.get("/books"), @app.post("/books"), etc.)**

  **>> Path parameters:** /books/{book_id} (dynamic part of URL)

  **>> Query parameters:** /books?search=python&limit=10 (optional filters after ?)

- **Request Object:**

When a client (browser, Swagger, or another app) calls your endpoint, FastAPI creates a Request object behind the scenes. This object contains:

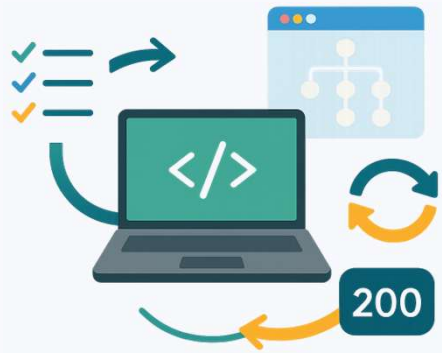
**>>** The **HTTP method** (**GET**, **POST**, **PUT**, **DELETE**, etc.)
**>>** The full URL and path
**>> Headers** (e.g., Authorization, Content-Type)
**>> Query parameters**
**>> Request body** (for POST/PUT)
**>>** Client information (IP address, user-agent, etc.)

You usually don't need to use the raw **Request** object in simple endpoints, but FastAPI gives you easy access to everything inside it through path parameters, query parameters, and Pydantic models.
Later in the course, you can inject **Request** directly if you need low-level access

**Request/Response Lifecycle ->**

## Request/Response Lifecycle:

1. Client sends an HTTP Request (method + path + headers + body/params)
2. Uvicorn (the ASGI server) receives the request and passes it to FastAPI
3. FastAPI matches the route and runs any dependencies
4. Your path operation function executes
5. FastAPI converts the return value to JSON (serialization) and adds the correct status code
6. The HTTP Response (with headers, body, and status code) goes back to the client




Status Codes ->

## Status Codes:

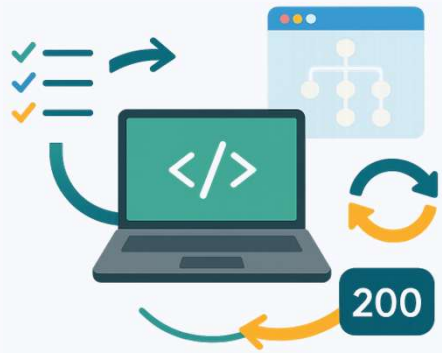
| Range | Category | Common Examples |
|---|---|---|
| **1xx** | Informational | 100 Continue, 101 Switching Protocols |
| **2xx** | Success | 200 OK, 201 Created, 204 No Content |
| **3xx** | Redirection | 301 Moved Permanently, 302 Found, 304 Not Modified |
| **4xx** | Client Error | 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable Entity |
| **5xx** | Server Error | 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable |

**Statuc Codes ->**

## Status Codes:

- **2xx Success:**
  - **>> 200 OK** – Standard success (default for GET)
  - **>> 201 Created** – Resource was successfully created (used for POST)
  - **>> 204 No Content** – Success but no body to return (often used for DELETE)

- **4xx Client Errors:**
  - **>> 400 Bad Request** – The request is invalid
  - **>> 401 Unauthorized** – Authentication required but missing or invalid
  - **>> 403 Forbidden** – Authenticated but not allowed
    (e.g., normal user trying to delete a book)
  - **>> 404 Not Found** – Resource does not exist
  - **>> 422 Unprocessable Entity** – Validation error (Pydantic rejected the data)




- **5xx Server Errors:**
  - **>> 500 Internal Server Error** –
    Something went wrong on the server (you usually don't raise this manually)

Always return the most appropriate status code — it helps clients (and frontends) understand what happened

**Task Time ->**

# Task Time

- In the **<<main.py>>:**

  **>>** Create a **books_db** local books storage that stores a list of books

  **>>** Create a **GET /books** route, that triggers an async function. The function should return the list of books. The status code will be 200

  **>>** Create a **POST /books** route, that triggers an async function. The function should take a Book Dictionary and adds it to the list of books. The status code will be 201

  **>>** Create a **GET /books/search** route, that triggers an async function. The function should take the book's title as an argument. The status code will be 200

  **>>** Create a **GET /books/{book_id}** route, that triggers an async function. The function should take the book's id as an argument. The status code will be 200. If the book is not found, rais an HTTPException with a 404 status code

Tasks Time ->

# Task Time

- In the **<<main.py>>:**

  **>>** Add PUT /books/{book_id} (status_code=200) and DELETE /books/{book_id} (status_code=204)

  Use **HTTPException(status_code=404)** when the book is not found

**Python Typing & Pydantic Models ->**

## Python Typing & Pydantic Models :

Type hints **(str**, **int**, **Optional[str])** help your editor and FastAPI understand expected data. **Pydantic models (BaseModel)** add runtime validation, automatic type conversion, default values, and clear error messages

- BookCreate for incoming data
- BookRead for outgoing data (can hide fields)

  **Serialization:** FastAPI + Pydantic turns models to JSON automatically

**Settings: BaseSettings** loads **.env** safely and validates config at startup

**Task Time ->**

# Task Time

1. Create the **<<schemas>>** folder and the **app folder**
2. Create the schemas folder a **<<book.py>>** file

**>>** Create a pydantic class BookBase BaseModel.

This class should take:

**-- Title** of a type string that is required with no default value with a min_length 1 and max_length 200

**-- Author** of a type string that is required with no default value with a min_length 1

**-- isbn** of a type string that is required with no default value with a min_length 10 and max_length 13

**-- description** of a type string optionnaly and takes a None as default value with a max_length 1000

**-- publication_year** of a type int optionnaly that is required with no default value with a value greater than or equal to 1500 and less then or equal to 2026

**>>** Prepare a pydantic class BookCreate BaseModel for later.

**>>** Create a pydantic class BookRead BaseModel, This class should take:

-- id of a type int

-- cover_image that could optionally be a string and by default is set to None

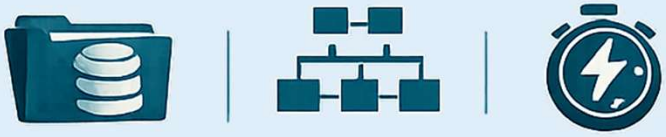
Task Time ->

# Task Time

- Refactor the POST endpoint to use **book: BookCreate** and **response_model=BookRead** Update in-memory storage to use model_dump()
- Add field constraints (e.g., ISBN regex) and test validation errors in Swagger.
- Extend search with more query params. Debug 422 errors together

Python Typing & Pydantic Models ->

# Structure, Data & Async



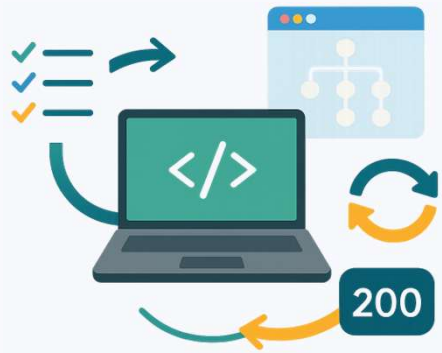
# FastAPI Essentials: Path Operations

- **APIRouter** lets you split code (books in one file, auth in another) and include them with prefixes/tags

- **Dependencies** (Depends()) are reusable functions FastAPI runs automatically before your endpoint. They provide DB sessions, current user, etc.

  Dependency Injection = FastAPI “injects” the result into your function parameters — clean and testable

- **Background Tasks** run after the response is sent (non-blocking)




Task Time ->

# Task Time

- Create a **<<router>>** folder in the app folder that have a **<<books.py>>** file
- Import **APIRouter** and **BackgroundTasks** from fastapit

- Create an **APIRouter** object that have a **prefix "/book**" and **tags "books"**

- Create an async function that take a message as parameter of a string type. This function should return a message that says **[BACKGROUND LOG] message**

- Create a POST / endpoint, that take as **response_model** the **BookRead** class.

This endpoint should trigger an async function. This function should take a book object of a type **BookCreate** and a **background_tasks** of a **BackgroundTasks** object.
This function will trigger the background task which is the previously created function and returns the book

**Task Time ->**

# Task Time

Create a simple pagination dependency in **app/dependencies.py**

- Add dependency for common query filters

Data Access with SQLModel ->

# Data Access with SQLModel:

SQLModel combines Pydantic validation with SQLAlchemy ORM. Define a class once — it becomes both a DB table and a data model.

Use **Session** to interact with the database (add, query, commit)

**Why It Matters?**

In-memory data disappears on restart. A real DB makes the library persistent

**Task Time ->**

# Task Time

- Create a **<<crud>>** folder in the app folder that have a **<<book.py>>** file
- Create a **<<core>>** folder in the app folder that have a **<<database.py>>** file
- Update the <<main.py>> accordingliny

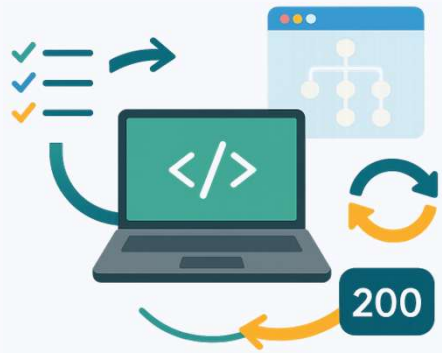
***---- This part will be done together in the class ----***

**Async Sessions & Migrations Basics ->**

## Async Sessions & Migrations Basics:

- **async def + await** allows the server to handle other requests while waiting for I/O (DB, file).
- **AsyncSession** from SQLModel/SQLAlchemy
- **Alembic** tracks schema changes safely (alembic revision –autogenerate

**Why It Matters?** Prepares the app for real traffic (many simultaneous users)




Task Time ->

# Task Time

- Update **app/core/database.py** with **async** engine and **sessionmaker**.
- Create dependency **get_async_session**
- Convert key endpoints to **async**
- Run first migration.

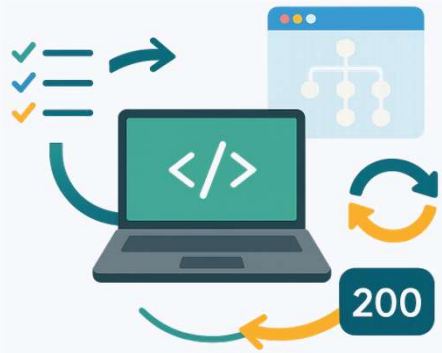
Full async CRUD for books.
Add a simple Book ↔ User relationship stub. Test migrations and endpoints in Swagger

Advanced Features, Security & Testing ->

# Advanced Features, Security & Testing

## Advanced Features, Security & Testing

- **UploadFile + File()** handles file uploads
- **StaticFiles** serves images
- **CORS** allows your future frontend to call the API
- **Middleware** runs on every request/response (logging, security headers)




**Why It Matters in BookLibrary?** Admins upload real cover images that users can see.

**Task Time ->**

# Task Time

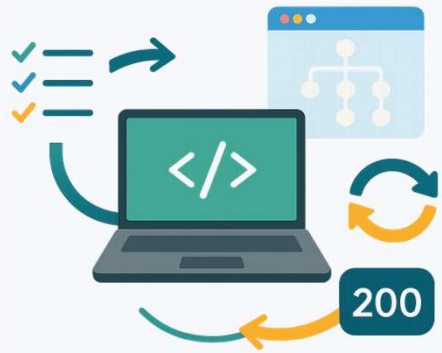
- Mount static folder and update **create_book** to accept and save **UploadFile**. Add **CORS** and a request-timing **middleware**.

Authentication & Authorization ->

## Authentication & Authorization

- **OAuth2 Password Flow** + password hashing (passlib[bcrypt]) for login
- **JWT** creates a secure token containing user info (expires, roles)
- Dependencies get_current_user and get_current_admin protect routes

**Why It Matters?** Only admins manage books; regular users search and borrow




Task Time ->

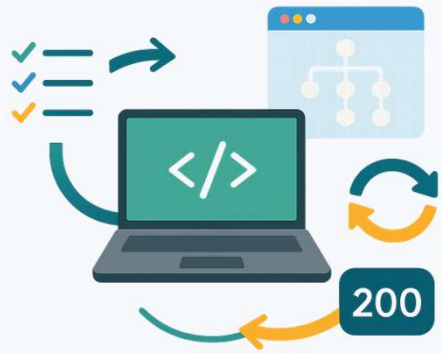
# Task Time

- Create app/models/user.py, app/core/security.py (hashing + JWT encode/decode), app/routers/auth.py (register + login)
- Add role field. Protect book CRUD except search. Implement borrow endpoint (protected + background task)

**Automatic API Docs, Versioning & Error Handling + Testing ->**

# Automatic API Docs, Versioning & Error Handling + Testing

- Swagger/ReDoc are auto-generated from your code and models
- Versioning (/api/v1/) keeps old clients happy
- Custom exception handlers give nice error messages.
- **Testin**g with pytest + httpx/TestClient simulates real calls and checks responses




Task Time ->

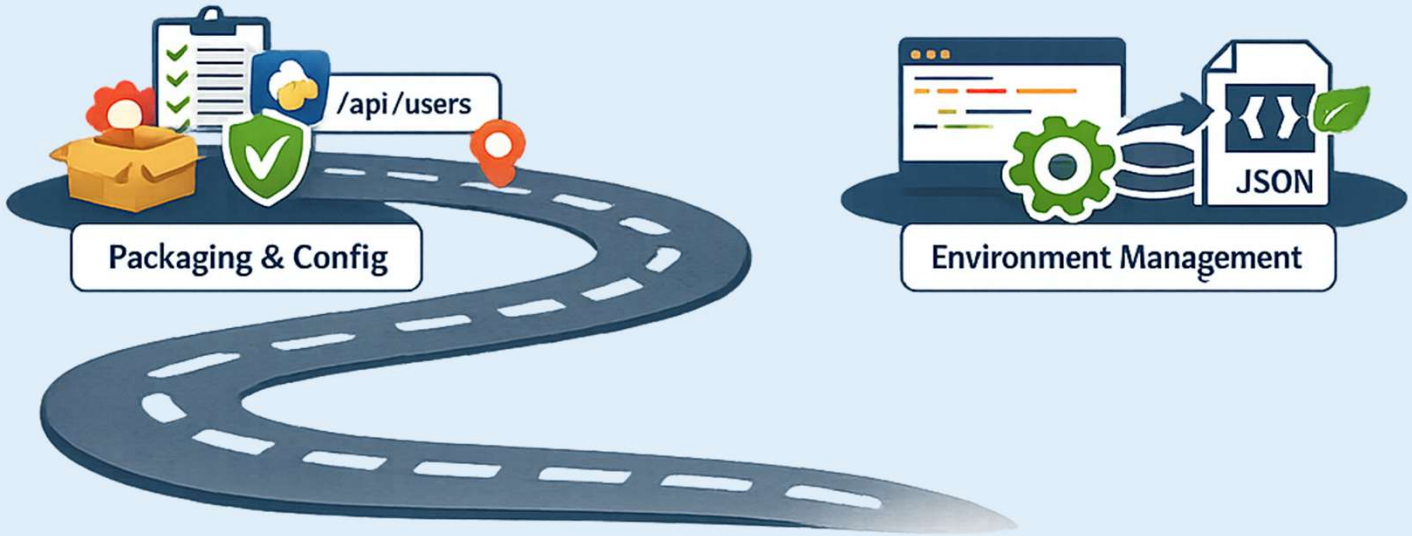
# Task Time

- Add /api/v1 prefix. Customize docs. Write tests for CRUD, auth, validation
- Run with pytest.

- Secure routes, upload a cover, write 6–8 tests, debug common JWT errors together

Production, Performance & Wrap-up ->

# Packaging, Configuration & Environment




## Depioying FastAPI

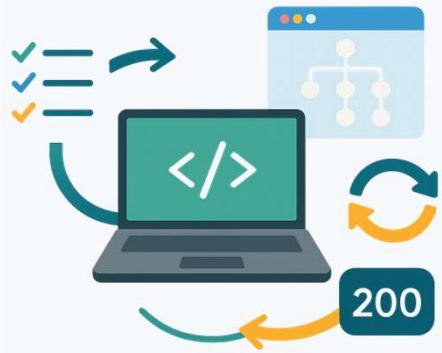
## Production, Performance & Wrap-up

In professional projects, we don’t install packages manually every time.

We define all dependencies in a structured way so the project is reproducible across machines and environments.

We use **pyproject.toml** (the modern Python standard) instead of a plain **requirements.txt**.

We already have **config.py** using **pydantic-settings**, which is the best practice for managing environment variables and secrets.
It automatically loads values from **.env** files and provides type validation




**Let’s Implement it in our project**

**Environment Management ->**

# Environment Management

We use **.env + pydantic-settings** (already implemented in config.py).

This allows us to have different settings for development and production without changing code.

Our current structure clearly separates:

- >> **config.py → configuration**
- >> **app/core/ → core utilities (database, security, middleware)**
- >> **app/models/ → database models**
- >> **app/schemas/ → request/response models**
- >> **app/crud/ → business logic**
- >> **app/routers/ → API endpoints**
- >> **app/dependencies.py → reusable dependencies**

This layered architecture makes the codebase maintainable and scalable.

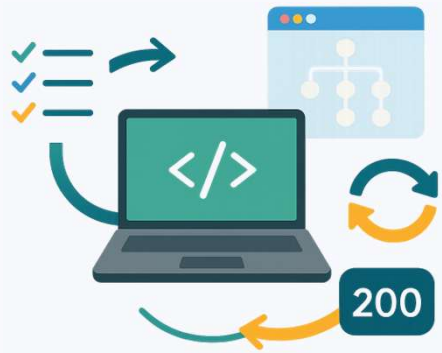
**Performance Basics – asyncio, concurrency, profiling & caching strategies ->**

# Performance Basics – asyncio, concurrency, profiling & caching strategies

- **Asyncio & Concurrency:** Our API already uses **async def** and **await** with **async database sessions**. This allows the server to handle many requests simultaneously without blocking while waiting for database operations.

- **Profiling:** Profiling helps identify which parts of the code are slow.

- **Caching Strategies:** Caching is one of the most effective performance improvements. Instead of hitting the database on every request, we store frequent results (especially search results) in memory




**Let's Implement this in our project**

**Profiling & Caching Strategies ->**

# Profiling & Caching Strategies

**What is Profiling?**
Profiling means **measuring how long parts of your code take to run** so you know where the slowness is. You can't fix a performance problem you haven't found yet.

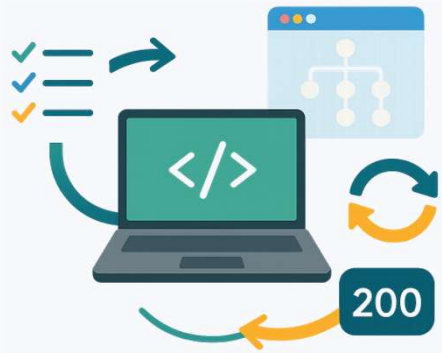
**SIMPLE ANALOGY**
Think of profiling like a doctor running tests before prescribing medicine. You don't guess — you measure first, then fix.

**Profiling Example ->**

## Profiling Example

- **Example 1 — measuring time manually**

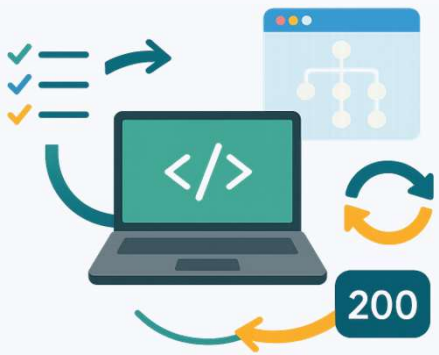
```
import time

def slow_function():
        total = 0
        for i in range(1_000_000):
                total += i
                return total

start = time.time()
slow_function()
end = time.time()

print(f"Took: {end - start:.4f} seconds")
```




**Profiling Example ->**

## Profiling Example

**EXAMPLE 2 — USING CPROFILE (BUILT INTO PYTHON)**




```
import cProfile

def add_numbers():
        return sum(range(1_000_000))

def main():
        add_numbers()

cProfile.run("main()")
```

**Caching with @lru_cache ->**

## Caching with @lru_cache

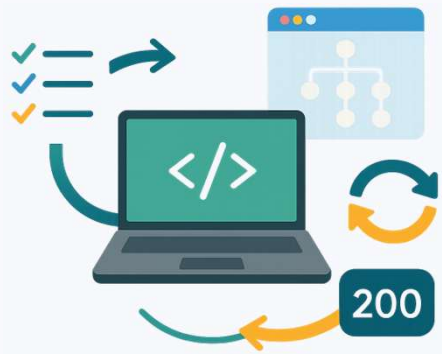
**@lru_cache** stores the result of a function in memory. The next time you call it with the same arguments, it returns the stored result instantly — no recomputation.

✕ WITHOUT CACHE

```
def fibonacci(n):
    if n ≤ 1:
        return n
    return (
        fibonacci(n-1) +
        fibonacci(n-2)
    )
# fibonacci(35) takes ~3 seconds
# recalculates everything
```

✅ WITH CACHE

```
from functools import lru_cache

@lru_cache(maxsize=128)
def fibonacci(n):
    if n ≤ 1:
        return n
    return (
        fibonacci(n-1) +
        fibonacci(n-2)
    )
# fibonacci(35) takes 0.0001s
```




Caching with @lru_cache ->

# Caching with @lru_cache

## CHECKING CACHE INFO

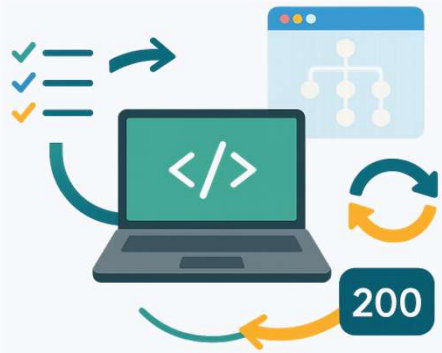
```python
from functools import lru_cache

@lru_cache(maxsize=128)
def square(n):
    print(f"Computing {n}²...")
    return n * n

square(4)   # computes
square(4)   # returns from cache instantly
square(5)   # computes

print(square.cache_info())
```

Caching with @lru_cache ->

# Caching with @lru_cache

**Clearing the cache.**

```
square.cache_clear()   # wipe everything stored in cache
print(square.cache_info())  # hits=0, misses=0
```




**Manual Dictionary Cache ->**

# Manual Dictionary Cache

When **@lru_cache** doesn't fit (e.g. async functions), a plain dictionary works perfectly as a cache.

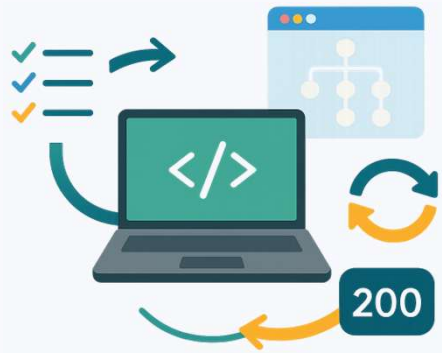
```python
# Simple dictionary cache
_cache = {}

def get_user(user_id: int):
    # Check cache first
    if user_id in _cache:
        print(f"[CACHE HIT] user {user_id}")
        return _cache[user_id]

    # Cache miss — simulate a DB call
    print(f"[CACHE MISS] fetching user {user_id} from DB...")
    user = {"id": user_id, "name": f"User {user_id}"}

    # Store in cache for next time
    _cache[user_id] = user
    return user

get_user(1)   # miss — goes to DB
get_user(1)   # hit  — returns instantly
get_user(2)   # miss — goes to DB
get_user(1)   # hit  — returns instantly
```

**Manual Dictionary Cache ->**

# Manual Dictionary Cache

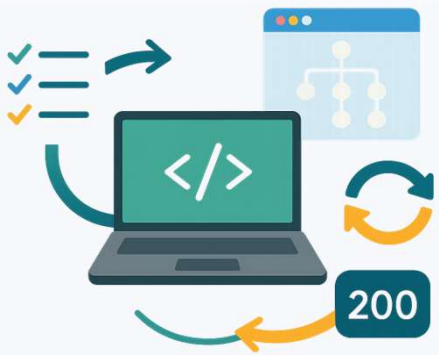
**Cache invalidation — clearing stale data**

```python
def update_user(user_id: int, new_name: str):
    # update in DB...
    print(f"Updated user {user_id} in DB")

    # Remove from cache so next call fetches fresh data
    if user_id in _cache:
        del _cache[user_id]
        print(f"[CACHE] Removed user {user_id} from cache")

get_user(1)              # miss
get_user(1)              # hit
update_user(1, "Alice")  # invalidates cache
get_user(1)              # miss again — fetches fresh
```




**When to Use What ->**

# When to Use What




**USE@LRU_CACHE WHEN: :**

- The function is **synchronous**
- The same inputs always give the same output
- The function is called repeatedly with the same arguments
- Examples: file paths, config values, computed constants

**Use a dict cache when:**

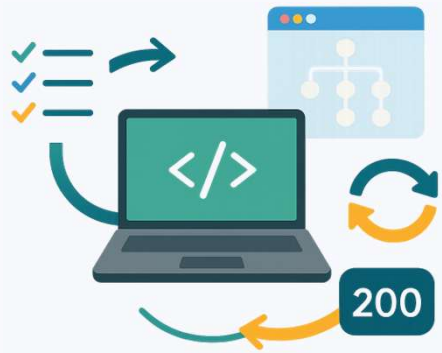
- You need to cache results of async functions
- You need full control over what gets cached and when
- You need to invalidate specific entries on update/delete
- Examples: DB query results, API responses

**When to Use What ->**

## When to Use What

**Use Redis when:**

- Your app runs on multiple servers
- Cache must survive server restarts
- Cache must be shared across processes
- Examples: production apps, scaled deployments




**Deploying FastAPI ->**

# Deploying FastAPI – Uvicorn/Gunicorn, systemd, reverse proxy, Docker container basics

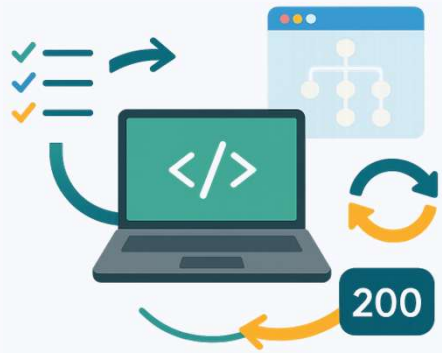
For production, we need a robust server setup:

**Uvicorn → good for development**
**Gunicorn + Uvicorn workers → production (multiple processes)**
**Docker → packages the entire application consistently**
**Reverse Proxy (Nginx/Caddy) → handles SSL, static files, and load balancing (concept only**

**Systemd ->**

# systemd

**What is it?**
systemd is the process manager built into Linux. It starts, stops, and monitors services on your server. Think of it as the server's task manager — it makes sure your app starts automatically on boot and restarts it if it crashes.




**without systemd:**
Server reboots → your app is dead, someone has to restart it manually

**with systemd:**
Server reboots → systemd → starts your app automatically
App crashes → systemd → restarts it automatically

create a service file ->

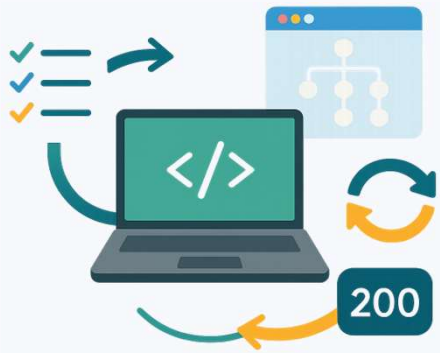
# Create a Service File

```
[Unit]
Description=BookLibrary FastAPI App
After=network.target          # start only after network is up

[Service]
User=ubuntu                   # the Linux user that runs the app
WorkingDirectory=/home/ubuntu/booklibrary
ExecStart=/home/ubuntu/booklibrary/venv/bin/gunicorn \
    --worker-class uvicorn.workers.UvicornWorker \
    --workers 4 \
    --bind 0.0.0.0:8000 \
    app.main:app
Restart=always                # restart if it crashes
RestartSec=3                  # wait 3 seconds before restarting

[Install]
WantedBy=multi-user.target    # start on normal system boot
```

**Enable and Start it ->**

## systemd

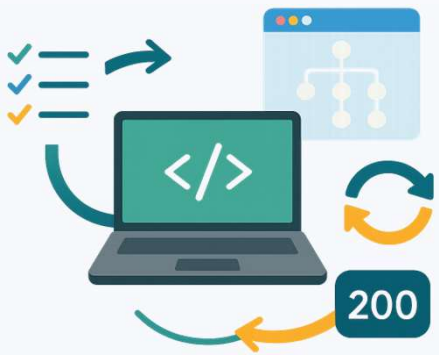
**sudo systemctl daemon-reload :** tell systemd to read the new file
**sudo systemctl enable booklibrary:** start automatically on boot
**sudo systemctl start booklibrary:** start it right now
**sudo systemctl status booklibrary:** check if it's running
**sudo systemctl stop booklibrary:** stop the app
**sudo systemctl restart booklibrary:** restart after a code change
**journalctl -u booklibrary –f:** watch live logs




**When to use systemd vs Docker?**
Use systemd when deploying directly on a Linux server (a VPS like DigitalOcean or AWS EC2). Use Docker when you want portability or are deploying to a container platform.

**Reverse Proxy ->**

# Reverse Proxy




**What is it?**
A reverse proxy is a server that sits in front of your app and handles incoming requests before passing them to it. Nginx is the most common choice.

User browser → Nginx (port 80/443) → Your FastAPI app (port 8000)
Nginx handles: HTTPS, domain name, static files, load balancing
Your app handles: API logic only

**Why not expose your app directly?**

❌ WITHOUT NGINX

Users hit port 8000 directly. No HTTPS, no domain name, no protection. Not how real websites work.

✅ WITH NGINX

Users hit port 80/443 with your domain. Nginx handles HTTPS and forwards cleanly to your app.

Reverse Proxy ->

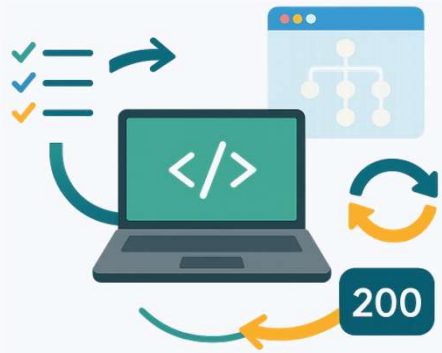
# Reverse Proxy

**STEP 1 — INSTALL NGINX:**

sudo apt update
sudo apt install nginx

**STEP 2 — CREATE A CONFIG FILE**

```
server {
    listen 80;
    server_name yourdomain.com;   # your domain or server IP

    location / {
        proxy_pass         http://127.0.0.1:8000;  # forward to your app
        proxy_set_header   Host $host;
        proxy_set_header   X-Real-IP $remote_addr;
    }

    # serve uploaded cover images directly (faster)
    location /static/ {
        alias /home/ubuntu/booklibrary/static/;
    }
}
```




Reverse Proxy ->

# Reverse Proxy

**Enable and restart:**

```
# link config to sites-enabled
sudo ln -s /etc/nginx/sites-available/booklibrary /etc/nginx/sites-enabled/

# test the config for errors
sudo nginx -t

# restart nginx
sudo systemctl restart nginx
```

**proxy_pass http://127.0.0.1:8000 —** this is the key line. It tells Nginx: "take every request that comes in and forward it to your FastAPI app running on port 8000 on the same machine."

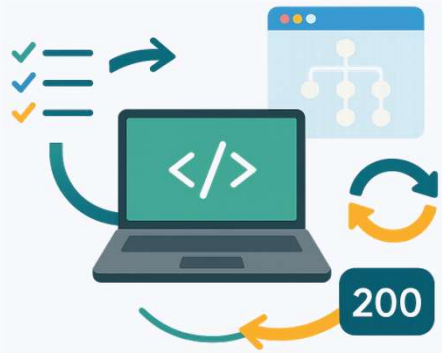
**Reverse Proxy ->**

# Reverse Proxy

**ADD HTTPS (OPTIONAL BUT RECOMMENDED)**

USE CERTBOT TO GET A FREE SSL CERTIFICATE AUTOMATICALLY:

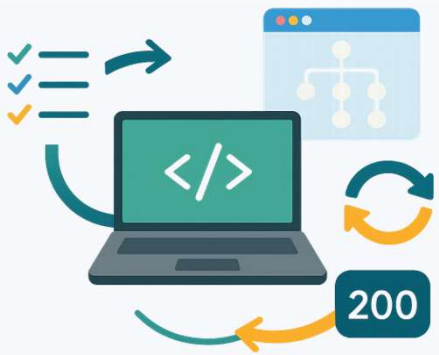
```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d yourdomain.com
# Certbot automatically updates your Nginx config for HTTPS
```




After this your app is reachable at **https://yourdomain.com instead of http://yourip:8000 —** that's the full production setup.

**Reverse Proxy ->**

# Reverse Proxy




**The Full Picture:**

**User visits https://yourdomain.com**
**↓**
**Nginx — handles HTTPS, domain, static files**
**↓ proxy_pass**
**Gunicorn + Uvicorn workers — runs your FastAPI app on port 8000**
**↓**
**systemd — keeps both Nginx and Gunicorn alive and auto-restarts them**
**↓**
**Your SQLite database**

**Reverse Proxy & systemd ->**

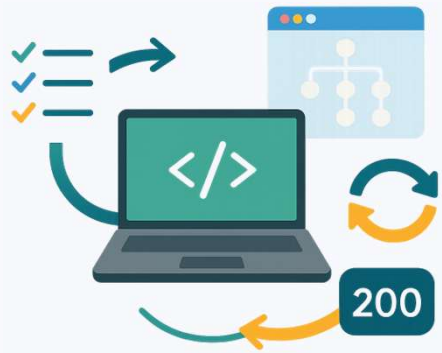
# Reverse Proxy & systemd

SYSTEMD HANDLES

Auto-start on boot, crash recovery, process monitoring, logs via journalctl

NGINX HANDLES

Domain routing, HTTPS/SSL, serving static files, forwarding requests to your app




Frameworks Comparison ->

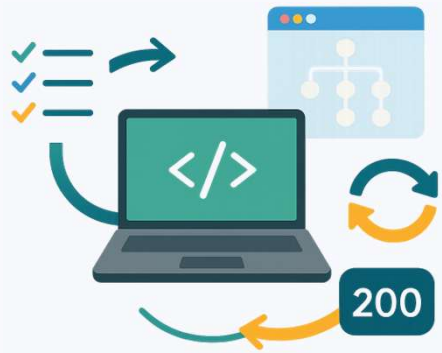
# Brief Comparison with Flask / Diango Rest Framework to Choose the Right Tool

Digital Learning Hub

## Comparison between Flask and Django

Now that you have built a complete API with FastAPI, it's important to understand when to use FastAPI versus other popular Python web frameworks.

Here’s an honest and practical comparison between **FastAPI**, **Flask**, and **Django** REST Framework




**Comparison ->**

# Comparison

| Aspect | FastAPI | Flask | Django REST Framework |
|---|---|---|---|
| **Performance** | Excellent (Async support built-in) | Good | Good |
| **Auto-generated Docs** | Best in class (Swagger UI + ReDoc) | Manual (requires Flask-RESTful or Swagger) | Good (Browsable API) |
| **Type Validation** | Excellent (Pydantic built-in) | Manual | Good (Serializers) |
| **Learning Curve** | Medium | Easiest | Steepest |
| **Development Speed** | Very Fast | Fast | Fast (once you know it) |
| **Async Support** | Native & Excellent | Limited (requires extensions) | Limited |
| **Built-in Admin Panel** | No (you build your own) | No | Yes (very powerful) |
| **Project Size** | Small to Large APIs | Small to Medium projects | Large, complex applications |
| **Boilerplate Code** | Very Low | Low | High |
| **Modern Python Features** | Excellent (type hints, async, Pydantic) | Basic | Good |
| **Community & Ecosystem** | Growing rapidly | Very mature | Very mature |
| **Best For** | Modern REST APIs, Microservices, High-performance backends | Small APIs, Prototypes, Simple services | Full-featured web applications with admin panel |

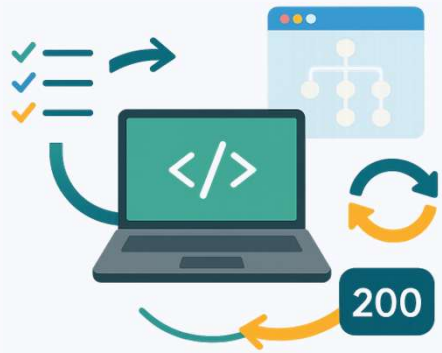
**FastAPI ->**

# FastAPI

FastAPI — Best Choice When:

- You want maximum performance and async support
- You need beautiful, automatic documentation (Swagger + ReDoc)
- You love modern Python (type hints, Pydantic)
- You're building REST APIs or microservicesYou want fast development with fewer bugs (thanks to validation)

Our BookLibrary API is a perfect example of what FastAPI excels at




Flask ->

## Flask

Flask — Best Choice When:

- You need a lightweight framework
- You're building a small prototype or simple API
- You want full control and flexibility
- You're doing something unconventional (not a standard CRUD API)F

lask is like a "Swiss Army knife" — simple and flexible, but you have to add many things yourself (validation, docs, auth, etc.).

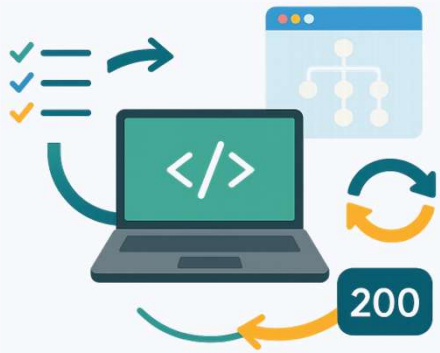
**Django REST Framework (DRF) ->**

# Django REST Framework (DRF)

Django REST Framework (DRF) — Best Choice When:

- You're building a large, complex web application
- You need a powerful Admin Panel out of the box
- You want many built-in features (authentication, permissions, pagination, filtering)
- You're working on a project that also needs traditional web pages (not just API)

Django is a "batteries-included" framework — very productive for big applications, but it can feel heavy for simple APIs.

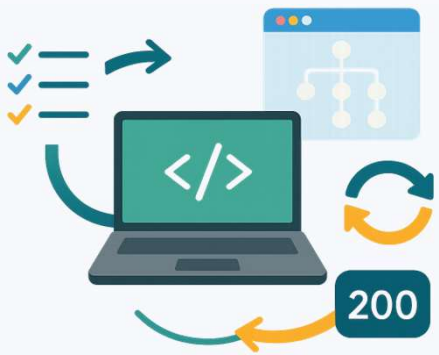



How to Choose the Right Tool ->

## How to Choose the Right Too

Use this decision guide:

- **Choose FastAPI if:**
  - >> You want high performance + great developer experience
  - >> You need excellent automatic documentation
  - >> You're comfortable with modern Python



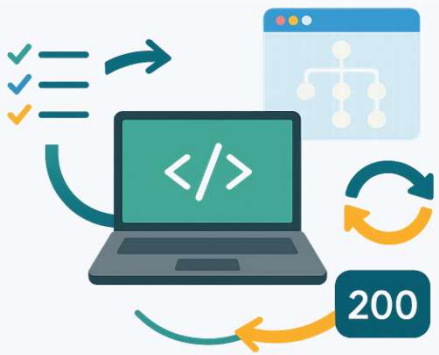

- **Choose Flask if**:
  - >> The project is small or a prototype
  - >> You want maximum flexibility
  - >> You prefer minimal framework

- **Choose Django REST Framework if:**
  - >> The project is large and complex
  - >> You need a built-in admin interface
  - >> You want many features without writing them from scratch

**FEEDBACK ->**

## Feedback

Leave us your feedback :
https://www.dlh.lu/survey/start/bf6edf45-ee16-4682-a972-42c914af7c35




Or by scanning the QR code